Rapport de Projet GPGPU : Méthodes de Monte-Carlo sur GPU (Version Allégée)

Master DSAD & IA — IUT de Douala

Membre : PIEGUE FOTSO DARWIN NATHAN

Date de remise : 10 Octobre 2026

Résumé

Ce projet présente l'implémentation et la parallélisation GPU d'algorithmes de Monte-Carlo pour l'estimation de π, la valorisation d'options financières européennes sous le modèle de Black-Scholes et l'échantillonnage par chaîne de Markov (Metropolis-Hastings). La logique des noyaux a été développée et validée localement sur simulateur Numba-CUDA avant d'être portée sur GPU physique via Google Colab.

1 Introduction et Contexte Mathématique

Les méthodes de Monte-Carlo s'appuient sur la Loi des Grands Nombres pour approximer des espérances mathématiques par des moyennes empiriques.

1.1 Convergence et Erreur Statistique

Soit $X_1, X_2, \ldots, X_N$ des variables aléatoires indépendantes et identiquement distribuées. L'estimateur de Monte-Carlo est :

$$\bar{X}_N = \frac{1}{N}\sum_{i=1}^{N} X_i \xrightarrow[N\to\infty]{\text{p.s.}} \mathbb{E}[X] \tag{1}$$

D'après le Théorème Central Limite, l'erreur statistique décroît selon $\mathcal{O}\left(\frac{1}{\sqrt{N}}\right)$.

1.2 Modèle de Black-Scholes

Le prix d'un actif financier S_t suit un Mouvement Brownien Géométrique :

$$dS_t = rS_t dt + \sigma S_t dW_t \tag{2}$$

La valeur exacte à maturité T pour un facteur d'incertitude $Z \sim \mathcal{N}(0,1)$ est :

$$S_T = S_0 \exp\left(\left(r - \frac{\sigma^2}{2}\right)T + \sigma\sqrt{T}Z\right) \tag{3}$$

La formule fermée analytique d'un Call européen sert de référence pour la validation numérique :

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{4}$$

où $d_1 = \frac{\ln(S_0/K)+(r+\sigma^2/2)T}{\sigma\sqrt{T}}$ et $d_2 = d_1 - \sigma\sqrt{T}$.

2 Architecture GPU Visée et Simulation Numba-CUDA

L'architecture cible est basée sur le modèle SIMT (Single Instruction, Multiple Threads) des GPU NVIDIA.

2.1 Distribution de la Charge

Chaque thread GPU exécute une boucle locale simulant N_{thread} trajectoires indépendantes en utilisant le générateur pseudo-aléatoire `xoroshiro128p_normal_float64`.

$$N_{\text{total}} = \text{Grille} \times \text{Blocs} \times N_{\text{thread}} \tag{5}$$

3 Validation Numérique et Résultats

Les implémentations CPU (NumPy) et GPU (Numba-CUDA Simulé) ont été comparées par rapport à la solution analytique exacte de Black-Scholes (Prix exact : 10.4506 €).

Tirages (N)	**Prix MC CPU (€)**	**Prix MC GPU Simulé (€)**	**Écart / Exact (€)**
10^3	10.7595	-	0.3089
10^5	10.4603	10.4502	0.0097
10^6	10.4372	-	0.0134

TABLE 1 – Validation de la convergence vers la solution analytique de Black-Scholes.

4 Étape Finale : Exécution sur GPU Réel (Colab / Kaggle)

L'étape finale a consisté à porter le code validé en simulateur sur un environnement cloud réel doté d'un accélérateur graphique NVIDIA Tesla T4 (via Google Colab). Les tests ont été menés à grande échelle sur un volume de $N = 100\,000\,000$ (10^8) trajectoires de Mouvement Brownien Géométrique.

Métrique	**Valeur mesurée (Tesla T4)**
Nombre de simulations (N)	100 000 000
Prix option estimé (€)	10.4502
Temps d'exécution total (s)	2.8111
Débit (Simulations / seconde)	35.57 Millions

TABLE 2 – Performances et métriques d'exécution sur GPU réel.

Les résultats confirment la pertinence de la parallélisation massive : l'absence de dépendance inter-threads permet d'exploiter pleinement les cœurs CUDA de la carte T4, rendant le calcul de 100 millions de scénarios financiers quasi instantané (moins de 3 secondes).

5 Conclusion

Les méthodes de Monte-Carlo sont naturellement adaptées à la parallélisation massive sur GPU en raison de l'absence de dépendance mémoire entre les tirages.

Références

[1] Black, F., & Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. *Journal of Political Economy.*

[2] Numba CUDA Documentation, `https://numba.readthedocs.io/`